WPT Day 3 Evening

Fri 23 Oct 26 · JavaScript as an object-oriented language: document / console / window, writing & running scripts, dynamic typing, operators (`==` vs `===`), template literals, conditionals, loops, functions, `var` / `let` / `const`, and hoisting — explained from first principles

---

Table of Contents

1. JavaScript as an Object-Oriented Language
2. Writing and Running JavaScript
3. Variables and Dynamic Typing
4. Operators
5. Template Literals
6. Control Flow: Conditionals
7. Loops and Nested Patterns
8. Functions
9. var, let, and const
10. Hoisting: How the JS Engine Reads Code
11. Combined Worked Example: Calculator with Patterns
12. Quick Revision Checklist
13. Practice Exercises
14. Practice Answers

JavaScript as an Object-Oriented Language

The problem this framing solves

Morning introduced JavaScript as the language of browser behaviour. Evening starts one level deeper: **almost everything you touch in the browser is an object**. Without that mental model, `document.write`, `console.log`, and `window.alert` look like three unrelated magic words. With it, they are the same pattern applied to three different objects.

Heavily inspired by Java — everything is an object

JS is heavily inspired by Java and treats **everything as an object**. That includes:

- Data values you invent (numbers, strings, booleans packaged as values you can inspect and transform)
- Visible UI: HTML elements, headings, tables, paragraphs, input fields, scrollbars
- Invisible machinery: fetching data from a server, sending data, dynamically updating the page

Not just what the user sees — **all operations act on objects**. Fetching, sending, rewriting the page mid-session: each of those is “ask an object to do something.”

Three fundamental predefined objects

The lecture introduces three objects you will use on every page:

Object	What it represents	What you use it for
`document`	The entire web page (the **DOM**)	Any operation on page content — write text, find elements, change HTML
`console`	The browser's developer tools console	Debug messages, errors, warnings — **not** visible to end users
`window`	The browser window itself	Window-level operations: scrollbar position, native popups, global scope

Background: **DOM** means Document Object Model — the browser's live tree of every node in the HTML page, exposed to JavaScript as the `document` object. You do not need the full DOM API yet; tonight's point is simply: the page *is* an object you can call methods on.

The syntax pattern: `object.method()`

Same dot-notation as Java: take an object reference, call a method on it.

```
document.write( ... )     →  ask the page to render text
console.log( ... )        →  ask the console to record a message
window.alert( ... )       →  ask the window to show a popup
```

Once you see the pattern, every new API in later sessions (`document.getElementById` , `window.addEventListener` , …) is the same shape with a different object and method name.

> **Exam:** Name the three fundamental predefined objects and one job each: `document` (page/DOM), `console` (dev-tools logging), `window` (browser window / popups / globals).

> **Pitfall:** Treating `console.log` as "something that appears on the page." It never does — only Inspect → Console shows it. End users of a normal site never see the console.

Writing and Running JavaScript

Where the code lives

JS code is written inside a `<script>` tag within the HTML file. There is **no compilation step**: the browser's JS engine reads the `<script>` tag and executes code **line by line immediately**. No separate compile button, no intermediate `.class` or `.o` file — open the HTML in a browser and it runs.

Placement: end of `<body>`, not `<head>`

Recommendation: put `<script>` at the end of `<body>`.

Placement	What happens
In `<head>`	Script may run **before** HTML elements exist → operations on those elements fail
End of `<body>`	Entire DOM is already parsed → elements exist when the script executes

Reason in one sentence: JS often operates on HTML elements; if the script runs before the HTML loads, those elements don't exist yet. Waiting until end of body ensures the entire DOM is available.

Background: Browsers parse HTML top-to-bottom. When they hit a `<script>` without special attributes, they pause HTML parsing, run the script, then continue. A script at the top that asks for a button that appears later in the file is asking for something that is not in the DOM yet.

Three output methods demonstrated

```
<!DOCTYPE html>
<html>
<head>
    <title>First JS</title>
</head>
<body>
    <h1>Demo page</h1>

    <script>
        document.write("Hello World");
        console.log("Hello Console");
        window.alert("Hi User");
    </script>
</body>
</html>
```

Method	Where output appears	Audience
`document.write("Hello World")`	Rendered into the **page body**	End user (visible on the page)
`console.log("Hello Console")`	Browser **dev tools** → Inspect → Console tab	Developer only
`window.alert("Hi User")`	Browser-native **popup dialog**	End user (blocking dialog)

Popup appearance varies across browsers (Chrome vs Edge vs Firefox) because the dialog belongs to the **browser window**, not the web page. Errors and warnings from JS also appear in the console area.

> **Exam:** Prefer script at **end of body** so the DOM exists. Three outputs: `document.write` (page), `console.log` (devtools), `window.alert` (native popup).

> **Pitfall:** Expecting `console.log` to show on the page, or expecting `alert` to look identical in every browser.

Variables and Dynamic Typing

The problem type declarations solve — and JS refuses to

In Java/C you write `int a = 4;` — the type is fixed at declaration. In JS there is **no explicit data type declaration**: simply name the variable and assign a value.

```
a = 4;          // no int, no String, no float
```

Type is decided at runtime

Type is determined **dynamically at runtime** based on the assigned value:

Assignment	Runtime type
`a = 4`	`number`
`a = "hello"`	`string`
`a = true`	`boolean`

A variable can **change its type mid-program** simply by reassigning a different kind of value — **no error is thrown**. That is what makes JS **not type-safe**: you cannot trust a variable to hold the same type throughout.

```
let x = 4;
console.log(typeof x);   // "number"
x = "hello";
console.log(typeof x);   // "string" – legal; no complaint
```

Checking type: `typeof`

```
typeof variableName
```

Returns a string such as `"number"`, `"string"`, `"boolean"`, `"object"`, `"undefined"`.

Special values: `null` vs `undefined`

Value	Meaning	`typeof` result
`null`	Explicitly assigned emptiness — intentional	`"object"` (known JS quirk)
`undefined`	Default of a declared-but-unassigned variable — absence of assignment	`"undefined"`

These two are **distinct**: `null` is intentional emptiness; `undefined` is "nothing was assigned."

```
let a;
console.log(a);           // undefined
console.log(typeof a);    // "undefined"

let b = null;
console.log(b);           // null
console.log(typeof b);    // "object"  ← quirk, not "null"
```

One numeric type: `number`

There is **no separate integer or float type**. All numeric values — whole or decimal — share the single type `number`.

```
typeof 5;     // "number"
typeof 5.0;   // "number"
```

> **Exam:** JS is dynamically typed / not type-safe; `typeof` reports the runtime type; `null` vs `undefined` differ (intentional vs unassigned); `typeof null === "object"` is a known quirk; integers and floats are both `number`.

> **Pitfall:** Assuming `typeof null` is `"null"`, or assuming `5` and `5.0` have different types.

Operators

Arithmetic

`+`, `-`, `*`, `/`, `%` (modulus), and `**` (exponentiation / power).

```
2 ** 4;   // 16
```

Relational — and the critical `==` vs `===` distinction

Operator	Compares	Type coercion?
`==` (loose equality)	Values only	**Yes** — coerces before comparing
`===` (strict equality)	Value **and** type	**No**
`≠`, `≠=`	Inequality counterparts	Same rules as `==` / `===`
`<`, `>`, `≤`, `≥`	Ordering	Usual numeric / string rules

Loose equality (`==`): when comparing a number with a string, JS extracts the numeric value from the string and then compares. The lecture calls this coercion **upcasting** in this context.

```
4 == "4";    // true  - "4" coerced to 4
4 === "4";   // false - number vs string
4 === 4.0;   // true  - both type number, same value
```

> **Exam:** Always prefer `===` in real code; seniors will flag `==` in code reviews.

> **Pitfall:** Writing `==` and being surprised when `"4" == 4` is `true`. Prefer `===` unless you *intentionally* want coercion.

String and arithmetic interaction

Expression	Result	Why
`"5" + 4`	`"54"`	`+` with a string **always concatenates**, never adds
`"5" - 4`	`1`	`-`, `*`, `/` **upcast** the string to its numeric equivalent
`"abc" - 4`	`NaN`	String cannot convert to a number

NaN (Not a Number) is the result of any arithmetic operation on invalid or non-numeric data. It is a common interview topic.

```
console.log("5" + 4);      // "54"
console.log("5" - 4);      // 1
console.log("abc" - 4);    // NaN
```

Logical operators

`||` (OR), `&&` (AND), `!` (NOT) — same purpose and behaviour as in Java/C.

Conditional (ternary) operator

Compact alternative to if-else for simple value assignment:

```
c = (a > b) ? a : b;   // larger of a or b
```

Syntax: `condition ? value_if_true : value_if_false`. No need to write a full if-else block for this kind of single-expression decision.

Template Literals

The readability problem

Traditional concatenation with `+` becomes unreadable when many variables are involved:

```
"Sum of " + a + " and " + b + " is " + c
```

Modern fix: backticks and `${}`

Template literals wrap the string in **backticks** (`` ` ``) — the key near Escape — and embed variables or expressions with `${…}`:

```
`Sum of ${a} and ${b} is ${c}`
```

Result is identical to concatenation but far easier to read and maintain. Supported by all current browsers. **Recommended approach going forward** whenever mixing strings and variables.

```
let a = 3, b = 7, c = a + b;
console.log(`Sum of ${a} and ${b} is ${c}`);
// Sum of 3 and 7 is 10
```

> **Exam:** Template literals use **backticks**, not quotes; embed with `${expression}`.

> **Pitfall:** Using single/double quotes and wondering why `${a}` prints literally as `${a}` — interpolation only works inside backticks.

Control Flow: Conditionals

Syntax familiar from Java/C

`if`, `if-else`, `if-else-if` ladder, nested `if`, and `switch-case` — syntax is identical to Java and C.

Deciding which to use

Situation	Prefer
Single condition (odd/even, positive/negative, voting eligibility)	`if-else`
Multiple mutually exclusive conditions	`if-else-if` ladder
Discrete known values of one expression	`switch-case`

The else-if chain **short-circuits**: once a true condition is found, remaining conditions are skipped entirely.

Lecture analogy: searching a line of students one by one — stop as soon as the match is found.

switch-case shape

- `switch` keyword, expression in parentheses
- Multiple `case` blocks, each ending with `break`

- A `default` block for no match

```
let day = 3;
switch (day) {
    case 1:
        console.log("Mon");
        break;
    case 2:
        console.log("Tue");
        break;
    case 3:
        console.log("Wed");
        break;
    default:
        console.log("Other");
}
```

Always use `===` inside conditions, not `==`, to avoid unintended type coercion bugs.

Loops and Nested Patterns

The three loop forms

Loop	When condition is checked	Notes
`while`	**Before** each iteration	Loop variable must be incremented manually or it runs infinitely
`for`	Init → condition → body → increment → condition…	Init, condition, and increment/decrement all in one line
`do-while`	**After** the body (at least one run)	Syntax same as Java/C; covered briefly

`for` execution cycle (same as Java/C):

```
initialization → condition check → body → increment → condition check → …
```

Nested loops: star pattern demonstration

- **Inner loop (`i`)**: prints one row of stars with `document.write("*")`
- After each row: `document.write("<br>")` breaks to the next line
- **Outer loop (`j`)**: repeats the inner loop for the desired number of rows

- For each iteration of `j`, the inner `i` loop runs completely (e.g. 5 stars × 10 rows)
- Increment order: after each star, `i` increments; only after the inner loop finishes does `j` increment

```
for (let j = 1; j ≤ 10; j++) {
    for (let i = 1; i ≤ 5; i++) {
        document.write("*");
    }
    document.write("<br>");
}
```

```
*****
*****
*****
… (10 rows of 5 stars)
```

Students are encouraged to re-implement all loop and condition programs from C/C++/Java in JS — **syntax is the only difference**.

> **Exam:** Nested pattern = outer loop for rows, inner for columns; `document.write("<br>")` after each row.

> **Pitfall:** Forgetting to increment the `while` loop variable → infinite loop. Forgetting `<br>` → one endless line of stars.

Functions

Why they exist

A function is a **named, reusable block of statements** grouped together. Two main benefits:

1. **Code reusability** — call the same logic multiple times without rewriting it
2. **Modularity** — break a large application into smaller, focused, independently readable pieces

Syntax differs from Java/C

Java/C	JavaScript
Return type required (`int` , `void` , …)	**No return type** (types don't exist at declaration time)

Java/C	JavaScript
Method inside a class	Standalone `function` keyword
Typed parameters	Arguments have **no type annotation**
Return often required by signature	`return` is **optional** depending on the requirement

```
function functionName(arg1, arg2) {
    // …
    return value;
}
```

Core concept (call, pass arguments, execute body, return value) is **identical** to Java/C — only the declaration syntax changed.

Single Responsibility Principle

Each function should do **exactly one thing**.

- An `add` function should add two numbers and **return** the result — nothing more
- What happens to the result afterward (log it, store it, send it to a server) is the responsibility of the **caller**, not the function
- Mixing responsibilities leads to tight coupling and harder debugging

```
function add(a, b) {
    return a + b;          // only job: add and return
}

let sum = add(3, 4);
console.log(sum);          // caller decides to log
document.write(sum);       // or write to page — still caller's choice
```

Overloading vs overriding

Feature	In JS?	Note
Function overloading	**Not supported**	No data types, no compile-time resolution
Workaround for “many args”	Variable-length arguments	Covered later
Method overriding	**Possible**	Covered later with classes and objects

Hoisting of function declarations

A function declaration can be **called before its definition** in the source code without error — because declarations are hoisted (full chapter below).

```
greet();   // works even though greet is defined below

function greet() {
    console.log("Hello");
}
```

> **Exam:** JS functions use `function` keyword, no return type, no typed args; SRP = one job; overloading not supported; function declarations are hoisted.

> **Pitfall:** Putting `console.log` inside `add` "for convenience" — that mixes logging into arithmetic and violates single responsibility.

---

var, let, and const

The problem: declaring with no keyword

Without any keyword, the variable **silently becomes a property of the `window` object**:

- Accessible everywhere in the program regardless of where it was declared (even inside a function)
- Cannot be called truly local or global — it is a **window-level property**
- This is a source of bugs; **always use one of the three keywords**

```
function oops() {
    x = 10;   // no keyword → window.x
}
oops();
console.log(window.x);   // 10 — leaked into the window
```

Comparison table

Rule	`var`	`let`	`const`
Scope	**Function**-scoped	**Block**-scoped (`{}`)	**Block**-scoped (same as `let`)

Rule	`var`	`let`	`const`
Redeclaration in same scope	Allowed (destroys old, creates new — no error)	**Not allowed** — `Identifier 'a' has already been declared`	**Not allowed**
Reassignment	Allowed	Allowed	**Not allowed** after initial assignment
Hoisting	Declaration hoisted; access before assignment → `undefined`	Hoisted into **temporal dead zone**; access before init → `ReferenceError`	Same TDZ rules as `let`
Modern recommendation	Avoid	Prefer for values that change	Prefer when value must never change

`var` in detail

- If declared outside a function → global; if inside → local to that function
- Accessible throughout the entire function, even from nested blocks (`if` , `for` , ...) within it
- Considered a **looser, less predictable** keyword; not recommended for modern code

```
function demoVar() {
    if (true) {
        var a = 1;
    }
    console.log(a);   // 1 - still visible; function scope, not block
}
```

`let` in detail

- Declared inside an `if` → only inside that `if` ; inside a loop → only inside that loop
- Preferred keyword for declaring variables in modern JS; widely used by seniors and in production

```
function demoLet() {
    if (true) {
        let a = 1;
    }
    // console.log(a);  // ReferenceError - a is not defined here
}
```

Accessing `let` before its initialization line throws:

```
Cannot access 'a' before initialization
```

Not `"a is not defined"`. That distinction confirms hoisting occurred (otherwise the error would be "not defined").

`const` in detail

Behaves identically to `let` for block scope, no redeclaration, and hoisting rules. **Only difference:** the value cannot be reassigned after the initial assignment. Equivalent to Java's `final`.

```
const PI = 3.14;
// PI = 3;   // TypeError - assignment to constant variable
```

Use `const` when the value should never change; use `let` when it needs to change.

> **Exam:** No-keyword → `window` property (bug magnet). `var` = function scope + redeclaration OK + pre-assign access is `undefined`. `let` / `const` = block scope + no redeclaration; `const` also forbids reassignment. Prefer `let` / `const` in modern code.

> **Pitfall:** Assuming `let` "isn't hoisted" because early access errors. It *is* hoisted — into the TDZ. The error text "Cannot access before initialization" is the proof.

Hoisting: How the JS Engine Reads Code

Mechanism

Before executing any JS code, the JS engine performs a **full read-through** of the entire source. During this read it identifies all `var` **declarations** and **function declarations**, then moves those declarations to the **top of their respective scope**. That process is called **hoisting**.

- Automatic — cannot be disabled
- Every JS engine does this

Practical effect for `var`

Developer writes:

```
console.log(a);
var a = 10;
```

JS engine effectively runs:

```
var a;           // declaration hoisted
console.log(a);  // undefined
a = 10;          // assignment stays in place
```

Result: `undefined` is logged, **not** an error.

Practical effect for `let` / `const`

Declaration is hoisted, but the variable is placed in a **temporal dead zone (TDZ)** until the initialization line is reached. Accessing it before that line throws:

```
ReferenceError: Cannot access 'a' before initialization
```

```
console.log(a);  // ReferenceError (TDZ)
let a = 10;
```

Function declarations

Hoisting also applies to function declarations, allowing functions to be called before they appear in the source (as shown in the Functions chapter).

```
Source order (what you write)          Engine's view after hoisting
──────────────────────────────         ──────────────────────────────
console.log(a);                        var a;
var a = 10;                            function greet() { … }
greet();                               console.log(a);   // undefined
function greet() { … }                 a = 10;
                                       greet();          // works
```

Exam: Hoisting = engine lifts `var` and function **declarations** to the top of scope before execution. `var` early access → `undefined`. `let` / `const` early access → TDZ `ReferenceError`.

Pitfall: Saying "`let` is not hoisted." It is hoisted into the TDZ; the error message differs from a truly undeclared name.

Combined Worked Example: Calculator with Patterns

One HTML file that ties together script placement, the three core objects, dynamic typing / `typeof`, `===`, template literals, conditionals, a nested star pattern, SRP-style functions, and `let` / `const` — open it in a browser as-is.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WPT Day 3 Evening — Combined Demo</title>
</head>
<body>
    <h1>Day 3 Evening Demo</h1>
    <p>Open DevTools (F12) → Console for logs. Popups and on-page stars appear below.
</p>
    <hr>
    <h2>Star pattern (5 × 4)</h2>

    <!— Script at end of body so the DOM above already exists —>
    <script>
        // --- const for fixed labels; let for values that change ---
        const APP_NAME = "Evening Demo";
        let a = 4;
        let b = "4";

        // --- Three fundamental objects ---
        window.alert("Welcome to " + APP_NAME);
        console.log("Console is for developers only");

        // --- typeof, == vs ===, null vs undefined ---
        let unset;
        let empty = null;
        console.log(`typeof a → ${typeof a}`);           // "number"
        console.log(`typeof b → ${typeof b}`);           // "string"
        console.log(`a == b  → ${a == b}`);              // true  (coercion /
"upcasting")
        console.log(`a === b → ${a === b}`);             // false (strict)
        console.log(`typeof unset → ${typeof unset}`);   // "undefined"
        console.log(`typeof empty → ${typeof empty}`);   // "object" (quirk)

        // --- String / arithmetic interaction + NaN ---
        console.log(`"5" + 4  → ${"5" + 4}`);            // "54"
        console.log(`"5" - 4  → ${"5" - 4}`);            // 1
        console.log(`"abc" - 4 → ${"abc" - 4}`);         // NaN

        // --- SRP functions: only compute; caller decides output ---
        function add(x, y) {
            return x + y;
```

```
        }

        function maxOf(x, y) {
            return (x > y) ? x : y;   // ternary
        }

        function describeNumber(n) {
            if (n > 0) {
                return "positive";
            } else if (n < 0) {
                return "negative";
            } else {
                return "zero";
            }
        }

        let sum = add(10, 20);
        let bigger = maxOf(10, 20);
        document.write(`<p>Sum of 10 and 20 is ${sum}</p>`);
        document.write(`<p>Larger is ${bigger}; 10 is ${describeNumber(10)}</p>`);

        // --- Nested loops: star pattern (outer = rows, inner = cols) ---
        const ROWS = 4;
        const COLS = 5;
        for (let j = 1; j ≤ ROWS; j++) {
            for (let i = 1; i ≤ COLS; i++) {
                document.write("*");
            }
            document.write("<br>");
        }

        // --- Hoisting demo (var): safe to read early → undefined ---
        console.log(`hoistedVar before assign → ${hoistedVar}`);
        var hoistedVar = 99;
        console.log(`hoistedVar after assign → ${hoistedVar}`);

        // greet is called before its declaration — function declarations hoist
        greet(APP_NAME);

        function greet(name) {
            console.log(`Hello from ${name}`);
        }
    </script>
</body>
</html>
```

Trace it end to end

1. **Parse order.** The browser builds the heading and paragraph first. Only then does it hit `<script>` at the end of `body`, so the DOM the script might need already exists (tonight we

mostly write into the page rather than query elements — the placement rule still holds for later DOM work).

2. **`window` / `console` / `document`.** `alert` is a window popup (looks different per browser). `console.log` never appears on the page — only in Inspect → Console. `document.write` injects HTML into the page body (sum line, larger line, stars).
3. **Dynamic typing & comparisons.** `a` is number `4`, `b` is string `"4"`. Loose `==` coerces and returns `true`; strict `===` sees different types and returns `false`. `typeof null` is `"object"` — the quirk the lecture flagged.
4. **`+` vs other operators.** `"5" + 4` concatenates; `"5" - 4` upcasts and subtracts; `"abc" - 4` yields `NaN`.
5. **SRP.** `add`, `maxOf`, and `describeNumber` only return values. The script (the caller) chooses `document.write` / template literals for display.
6. **Nested pattern.** Outer `j` = rows, inner `i` = stars per row, `<br>` after each row → 4 lines of 5 asterisks.
7. **Hoisting.** `var hoistedVar` can be read before its assignment line → `undefined`, then `99`. `greet` works before its source line because function declarations are hoisted in full.

If you can explain each of those seven steps without looking back, the evening's material is under control.

Quick Revision Checklist

Topic	One-line takeaway
JS as OO	Heavily Java-inspired; **everything is an object** — UI and invisible ops alike
`document`	The whole page (DOM); any page-content operation
`console`	DevTools console — debug only; not for end users
`window`	Browser window — popups, scroll, globals
Dot syntax	`object.method()` — same pattern as Java
Script placement	End of `<body>` so the DOM exists before JS runs
No compile	Browser engine runs `<script>` line by line immediately
Three outputs	`document.write` (page) · `console.log` (devtools) · `window.alert` (popup)

Topic	One-line takeaway
Dynamic typing	No type in declaration; type follows the assigned value; can change mid-program
Not type-safe	You cannot trust a variable to keep one type
`typeof`	Returns `"number"`, `"string"`, `"boolean"`, `"object"`, `"undefined"`, …
`null` vs `undefined`	Intentional empty vs never assigned; `typeof null` is `"object"` (quirk)
Numeric type	Only `number` — no separate int/float
`**`	Exponentiation: `2 ** 4 === 16`
`==` vs `===`	Loose (coerces / "upcasting") vs strict (value + type); prefer `===`
`"5" + 4`	`"54"` — `+` with a string always concatenates
`"5" - 4`	`1` — other arithmetic upcasts the string
`NaN`	Result of invalid numeric arithmetic; interview favourite
Ternary	`cond ? ifTrue : ifFalse` — compact if-else for one expression
Template literals	Backticks + `${expr}`; preferred over long `+` chains
if / switch	Same as Java/C; else-if short-circuits (student-line analogy)
Loops	`while` / `for` / `do-while`; nested stars = outer rows, inner cols + ` `
Functions	`function name(args) { … }`; no return type; return optional
SRP	One job per function; caller decides logging / storage / UI
Overloading	Not supported in JS; overriding comes later with classes
No keyword	Variable becomes a `window` **property** — always use `var` / `let` / `const`
`var`	Function scope; redeclaration OK; early access → `undefined`
`let`	Block scope; no redeclaration; TDZ → "Cannot access before initialization"
`const`	Like `let` + no reassignment (≈ Java `final`)
Hoisting	Engine lifts `var` + function declarations before run; `let` / `const` TDZ

Practice Exercises

Work these without looking at the answers. They target the exact confusions the lecture called out.

Concepts

1. Name the three fundamental predefined objects and give one concrete use of each.
2. Why should `<script>` sit at the end of `<body>` rather than in `<head>`?
3. A teammate says "`console.log` will show a welcome message to our users." Correct them in one sentence.
4. What does it mean that JavaScript is "not type-safe"? Give a two-line code example that proves it.
5. Distinguish `null` and `undefined`. What does `typeof` return for each, and which result is the famous quirk?

Operators and strings

6. Predict each result and explain in one phrase: (a) `4 == "4"` (b) `4 === "4"` (c) `4 === 4.0` (d) `"5" + 4` (e) `"5" - 4` (f) `"abc" - 4`.
7. Why do seniors flag `==` in code review? What should you write instead inside conditions?
8. Rewrite `"Sum of " + a + " and " + b + " is " + c` using a template literal.
9. Using the ternary operator only (no `if`), assign to `c` the larger of `a` and `b`.

Control flow, loops, functions

10. When do you choose `if-else` vs an `if-else-if` ladder? What does "short-circuit" mean for the ladder, and what analogy did the lecture use?
11. Sketch (in words or code) a nested loop that prints 3 rows of 7 stars using `document.write`, including the line break.
12. Write a function `multiply(x, y)` that obeys the Single Responsibility Principle. Then show how the *caller* might log the result without putting `console.log` inside `multiply`.
13. True or false, with justification: "JavaScript supports function overloading the same way Java does."

var / let / const and hoisting

14. What happens if you assign `x = 5` inside a function with **no** `var` / `let` / `const`? Why is that dangerous?
15. Fill the table mentally: for `var`, `let`, and `const` — scope kind, redeclaration allowed?, reassignment allowed?

16. Predict the output / error: `console.log(a); var a = 10;`
17. Predict the output / error: `console.log(a); let a = 10;` Explain how the error message proves hoisting still happened.
18. Spot the bug: a developer says "`let` is not hoisted, that's why early access fails." What is wrong with that statement?

Practice Answers

1. **`document`** — the page/DOM (e.g. `document.write`); **`console`** — DevTools logging (e.g. `console.log`); **`window`** — browser window (e.g. `window.alert`, scrollbar, globals).
2. JS often operates on HTML elements. In `<head>`, the script may run before those elements exist. At the **end of** `<body>`, the entire DOM is available when the script executes.
3. **`console.log` is only for developers** in Inspect → Console; end users never see it. Use `document.write` / DOM updates or UI text for users.
4. A variable's type can change mid-program with no error — you cannot trust it to stay one type. Example:

```
let x = 4;        // number
x = "hello";      // now string – legal in JS
```

5. **`null`** = intentionally assigned emptiness; **`undefined`** = declared but never assigned. `typeof undefined` → `"undefined"`; `typeof null` → `"object"` (the quirk).
6. (a) **`true`** — loose equality coerces `"4"` to `4`. (b) **`false`** — strict; different types. (c) **`true`** — both `number`, same value. (d) **`"54"`** — `+` concatenates with a string. (e) **`1`** — `-` upcasts `"5"` to number. (f) **`NaN`** — `"abc"` cannot become a number.
7. `==` performs **type coercion** ("upcasting"), which hides bugs. Prefer `===` (value and type, no coercion).
8. `` `Sum of ${a} and ${b} is ${c}` ``
9. `c = (a > b) ? a : b;`
10. **Single** condition → `if-else`. **Multiple mutually exclusive** conditions → `if-else-if`. Short-circuit = once a true branch runs, the rest are **skipped**. Analogy: searching a line of students one by one — stop when the match is found.
11. Outer loop rows 1…3; inner loop columns 1…7 writing `"*"`; after each inner loop, `document.write("<br>")`.

```
for (let j = 1; j ≤ 3; j++) {
    for (let i = 1; i ≤ 7; i++) {
        document.write("*");
    }
    document.write("<br>");
}
```

12. **SRP version and caller logging:**

```
function multiply(x, y) {
    return x * y;
}

let product = multiply(6, 7);
console.log(product);   // caller's job
```

13. **False.** Function overloading is **not supported** (no data types / no compile-time resolution). A variable-length-arguments workaround comes later; overriding comes with classes.
14. `x` becomes a **property of** `window`, visible everywhere — not truly local. That leaks state and causes hard-to-find bugs; always use `var` / `let` / `const`.
15. `var`**:** function scope; redeclaration yes; reassignment yes. `let`**:** block scope; redeclaration no; reassignment yes. `const`**:** block scope; redeclaration no; reassignment **no**.
16. Logs `undefined` (declaration hoisted; assignment still below), not an error.
17. `ReferenceError: Cannot access 'a' before initialization` (TDZ). If `a` were not hoisted at all, the error would be **"a is not defined"** — the actual message proves the binding exists but is uninitialized.
18. `let` ***is*** **hoisted** — into the temporal dead zone. Early access fails *because* of the TDZ, not because hoisting was skipped. The "Cannot access before initialization" wording is the evidence.